

OWASP Top 10 2021 Web Application Security Vulnerabilities

Security Research Report

Executive Summary

The Open Web Application Security Project (OWASP) Top 10 represents the most critical web application security risks based on data from security professionals worldwide. The 2021 edition reflects current threat landscapes and provides organizations with a foundation for developing secure applications. This report explores all ten vulnerabilities and provides detailed analysis of five critical risks.

Overview of OWASP Top 10 2021

The OWASP Top 10 2021 list includes the following vulnerabilities, ranked by their prevalence and potential impact:

1. A01:2021 – Broken Access Control

- **Risk Level:** Critical
- **Prevalence:** Found in 3.81% of tested applications
- **Overview:** Failure to properly restrict what authenticated users can do, allowing attackers to access unauthorized functionality or data.

2. A02:2021 – Cryptographic Failures

- **Risk Level:** High
- **Previously:** Known as "Sensitive Data Exposure" in 2017
- **Overview:** Failures related to cryptography that lead to sensitive data exposure or system compromise.

3. A03:2021 – Injection

- **Risk Level:** High
- **Prevalence:** Found in applications with 274k occurrences of CWEs
- **Overview:** Untrusted data sent to interpreters as part of commands or queries, including SQL, NoSQL, OS, and LDAP injection.

4. A04:2021 – Insecure Design

- **Risk Level:** High
- **New Category:** Added in 2021
- **Overview:** Flaws in design and architectural patterns that cannot be fixed by implementation alone.

5. A05:2021 – Security Misconfiguration

- **Risk Level:** Medium-High
- **Prevalence:** Most common vulnerability type
- **Overview:** Missing security hardening, improper configurations, or default settings left unchanged.

6. A06:2021 – Vulnerable and Outdated Components

- **Risk Level:** Medium-High
- **Previously:** Known as "Using Components with Known Vulnerabilities"
- **Overview:** Use of components with known vulnerabilities or outdated versions.

7. A07:2021 – Identification and Authentication Failures

- **Risk Level:** Medium
- **Previously:** Known as "Broken Authentication"
- **Overview:** Failures in user identity confirmation, authentication, and session management.

8. A08:2021 – Software and Data Integrity Failures

- **Risk Level:** Medium
- **New Category:** Added in 2021
- **Overview:** Code and infrastructure that do not protect against integrity violations.

9. A09:2021 – Security Logging and Monitoring Failures

- **Risk Level:** Medium
- **Previously:** Known as "Insufficient Logging & Monitoring"
- **Overview:** Insufficient logging, detection, monitoring, and active response capabilities.

10. A10:2021 – Server-Side Request Forgery (SSRF)

- **Risk Level:** Medium
 - **New Category:** Added in 2021
 - **Overview:** Fetching remote resources without validating user-supplied URLs.
-

Detailed Analysis of Five Critical Vulnerabilities

1. A01:2021 – Broken Access Control

Description

Broken Access Control occurs when applications fail to properly restrict what authenticated users are allowed to do. This vulnerability has moved from fifth position in 2017 to the top position in 2021, indicating its increasing prevalence and severity.

How It Works

Access control enforces policies to prevent users from acting outside their intended permissions. When these controls fail, attackers can:

- Access unauthorized functionality and/or data
- View sensitive files
- Modify or delete data
- Perform business functions outside their limits

Common Scenarios

- **Vertical Privilege Escalation:** Regular users gaining admin privileges
- **Horizontal Privilege Escalation:** Users accessing other users' accounts
- **Missing Function Level Access Control:** Accessing admin functions through direct URL manipulation
- **Insecure Direct Object References:** Changing parameter values to access unauthorized data

Real-World Examples

Scenario 1: URL Manipulation

Normal user URL: `/account/viewProfile?userid=123`

Attacker changes to: `/account/viewProfile?userid=124`

Result: Access to another user's profile

Scenario 2: Privilege Escalation

Normal function: `/user/changePassword`

Admin function: `/admin/deleteUser`

Attacker directly accesses admin function without proper authorization

Prevention Strategies

- Implement deny-by-default access controls
- Use centralized access control mechanisms
- Log access control failures and alert administrators
- Rate limit API access to minimize automated attack tool damage
- Invalidate JWT tokens on logout
- Implement proper session management

Detection Methods

- Code review focusing on access control logic
 - Automated testing of access control mechanisms
 - Manual testing of privilege escalation scenarios
 - Regular security assessments and penetration testing
-

2. A02:2021 – Cryptographic Failures

Description

Cryptographic Failures, previously known as "Sensitive Data Exposure," represents a shift in focus from the symptom to the root cause. This vulnerability occurs when applications fail to adequately protect data in transit and at rest through proper cryptographic controls.

How It Works

Many web applications and APIs do not properly protect sensitive data such as:

- Financial information
- Healthcare data
- Personal identifiable information (PII)
- Authentication credentials
- Private data requiring legal protection

Common Scenarios

- **Weak Encryption Algorithms:** Using outdated algorithms like MD5, SHA1, or DES
- **Poor Key Management:** Hard-coded keys, weak key generation, or improper key storage
- **Inadequate Data Classification:** Not identifying what data needs protection
- **Missing Encryption:** Transmitting or storing sensitive data in plain text
- **Downgrade Attacks:** Forcing use of weaker cryptographic protocols

Real-World Examples

Scenario 1: Weak Password Storage

Bad: Storing passwords with MD5 hashing

Better: Using bcrypt, scrypt, or Argon2

Scenario 2: Unencrypted Database

Database contains credit card numbers in plain text

Attacker gains database access → immediate data breach

Scenario 3: HTTP Usage for Sensitive Data

Login form submitted over HTTP instead of HTTPS

Credentials intercepted through man-in-the-middle attacks

Prevention Strategies

- Classify data and apply controls according to classification
- Use strong, up-to-date cryptographic algorithms
- Implement proper key management practices
- Encrypt all data in transit with secure protocols (TLS 1.2+)
- Encrypt sensitive data at rest
- Use authenticated encryption when applicable
- Store passwords using strong adaptive hashing functions

Technical Implementation

Proper Password Hashing (Example):

// Bad

```
password_hash = md5(password)
```

// Good

```
password_hash = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt())
```

TLS Configuration:

// Minimum TLS 1.2

// Disable weak cipher suites

// Implement HSTS headers

3. A03:2021 – Injection

Description

Injection flaws occur when untrusted data is sent to an interpreter as part of a command or query. Hostile data can trick interpreters into executing unintended commands or accessing data without authorization. Despite being third in 2021, injection remains one of the most dangerous vulnerabilities.

Types of Injection Attacks

- **SQL Injection:** Malicious SQL code execution
- **NoSQL Injection:** Targeting NoSQL databases
- **LDAP Injection:** Exploiting LDAP queries
- **OS Command Injection:** Executing operating system commands
- **XPath Injection:** Manipulating XML data queries

How SQL Injection Works

SQL injection occurs when user input is directly concatenated into SQL queries without proper validation or parameterization.

```
sql

-- Vulnerable Query
String query = "SELECT * FROM users WHERE username = '" + username + "' AND password = '" + password + "'"

-- Attack Input
username: admin'--
password: anything

-- Resulting Query
SELECT * FROM users WHERE username = 'admin'--' AND password = 'anything'
-- The '--' comments out the password check, allowing bypass
```

Real-World Attack Scenarios

Scenario 1: Data Extraction

Input: ' UNION SELECT credit_card_number FROM customers--

Result: Extracts all credit card numbers from database

Scenario 2: Data Modification

Input: '; UPDATE users SET password='hacked123' WHERE username='admin'--

Result: Changes admin password

Scenario 3: System Compromise

Input: '; EXEC xp_cmdshell('net user hacker password123 /add')--

Result: Creates new system user account

Prevention Strategies

- Use parameterized queries/prepared statements
- Implement strict input validation
- Use stored procedures (when properly implemented)
- Escape all user-supplied input
- Implement least privilege database access
- Use allowlist input validation
- Apply WAF (Web Application Firewall) rules

Code Examples

python

Vulnerable Code

```
cursor.execute("SELECT * FROM users WHERE id = " + user_id)
```

Secure Code - Parameterized Query

```
cursor.execute("SELECT * FROM users WHERE id = %s", (user_id,))
```

Secure Code - Prepared Statement (Java)

```
String sql = "SELECT * FROM users WHERE id = ?";
```

```
PreparedStatement pstmt = connection.prepareStatement(sql);
```

```
pstmt.setInt(1, userId);
```

```
ResultSet rs = pstmt.executeQuery();
```


4. A04:2021 – Insecure Design

Description

Insecure Design is a new category added in 2021 that represents missing or ineffective control design. It's distinct from insecure implementation - you can't fix insecure design with a perfect implementation. Secure design requires constant evaluation of threats and ensures code is designed and tested to prevent known attack methods.

Key Concepts

- **Threat Modeling:** Systematic approach to identifying and mitigating security threats
- **Secure Design Patterns:** Proven architectural solutions to common security problems
- **Defense in Depth:** Multiple layers of security controls
- **Principle of Least Privilege:** Minimal necessary access rights

Common Insecure Design Patterns

- Missing rate limiting on critical functions
- Lack of proper business logic validation
- Insufficient separation of concerns
- Missing security controls in the architecture
- Inadequate threat modeling during design phase

Real-World Examples

Scenario 1: Password Recovery Flaw

Design Issue: Password recovery only asks for username

Attack: Attacker can reset any user's password

Secure Design: Multi-factor authentication for password recovery

Scenario 2: Business Logic Bypass

Design Issue: Discount validation only on client-side

Attack: User manipulates request to get 100% discount

Secure Design: Server-side validation of all business rules

Scenario 3: Missing Rate Limiting

Design Issue: No limits on login attempts or API calls

Attack: Brute force attacks, resource exhaustion

Secure Design: Implement rate limiting, account lockouts, CAPTCHA

Prevention Through Secure Design

- Establish secure development lifecycle with security professionals
- Implement threat modeling for critical authentication, access control, and business logic flows
- Integrate security language and controls into user stories
- Implement tier-appropriate validation on all tiers
- Write unit and integration tests to validate critical flows
- Segregate tier layers on network and system levels

Design Principles

- **Fail Securely:** System should fail to a secure state
- **Default Deny:** Access should be denied by default
- **Complete Mediation:** Every access must be checked
- **Open Design:** Security should not depend on secrecy
- **Separation of Privilege:** Critical operations require multiple conditions

5. A05:2021 – Security Misconfiguration

Description

Security Misconfiguration is the most commonly seen issue and often results from using default configurations, incomplete or ad hoc configurations, open cloud storage, misconfigured HTTP headers, and verbose error messages containing sensitive information.

Common Misconfiguration Types

- **Default Accounts and Passwords:** Using vendor default credentials
- **Directory Listing:** Exposing sensitive files and directories
- **Excessive Error Information:** Revealing stack traces and system information
- **Missing Security Headers:** Lack of protective HTTP headers
- **Outdated Software:** Running unpatched systems and applications

Attack Scenarios

Scenario 1: Default Credentials

Default admin account: admin/admin

Attacker gains full system access using defaults

Scenario 2: Directory Traversal

Misconfigured web server allows:

`http://example.com/../../etc/passwd`

Result: System file exposure

Scenario 3: Detailed Error Messages

Database error reveals:

"SQL Server Exception: Invalid column 'credit_card' in table 'customers'"

Attack: Information disclosure about database structure

Cloud Security Misconfigurations

- Open S3 buckets with public read/write access
- Misconfigured IAM policies with excessive permissions
- Unencrypted data storage
- Missing network access controls
- Inadequate logging and monitoring

Prevention Strategies

- Implement repeatable hardening processes
- Remove unnecessary features, components, and documentation
- Regularly review and update configurations
- Use automated security configuration scanning
- Implement network segmentation
- Send security directives to clients (e.g., Security Headers)
- Establish automated process to verify configurations across environments

Security Headers Implementation

http

Essential Security Headers

Strict-Transport-Security: max-age=31536000; includeSubDomains

Content-Security-Policy: default-src 'self'

X-Frame-Options: DENY

X-Content-Type-Options: nosniff

Referrer-Policy: strict-origin-when-cross-origin

Permissions-Policy: geolocation=(), camera=(), microphone=()

Configuration Checklist

- ☐ Default passwords changed
- ☐ Unnecessary services disabled
- ☐ Security headers implemented
- ☐ Error handling configured securely
- ☐ File permissions properly set
- ☐ Regular security updates applied
- ☐ Network access controls in place
- ☐ Logging and monitoring enabled

Impact Assessment

Business Impact

- **Data Breaches:** Exposure of sensitive customer and business data
- **Financial Loss:** Regulatory fines, legal costs, business disruption
- **Reputation Damage:** Loss of customer trust and brand value
- **Compliance Violations:** GDPR, PCI DSS, HIPAA penalties

Technical Impact

- **System Compromise:** Full server or application takeover
 - **Data Integrity Loss:** Modification or destruction of critical data
 - **Service Availability:** Denial of service or system outages
 - **Lateral Movement:** Attackers moving through network infrastructure
-

Mitigation Strategies

Organizational Approaches

1. **Security Training:** Regular developer and security team education
2. **Secure SDLC:** Integration of security throughout development lifecycle
3. **Code Reviews:** Manual and automated security code analysis
4. **Penetration Testing:** Regular security assessments
5. **Incident Response:** Prepared response procedures for security events

Technical Controls

1. **Input Validation:** Comprehensive validation of all user inputs
 2. **Access Controls:** Proper authentication and authorization mechanisms
 3. **Encryption:** Strong cryptographic protection for sensitive data
 4. **Security Headers:** Implementation of protective HTTP headers
 5. **Monitoring:** Continuous security monitoring and logging
-

Tools and Resources

Static Analysis Tools

- SonarQube
- Checkmarx
- Veracode
- Fortify Static Code Analyzer

Dynamic Testing Tools

- OWASP ZAP
- Burp Suite Professional
- Nessus
- Acunetix

Security Frameworks

- NIST Cybersecurity Framework
- ISO 27001
- OWASP Application Security Verification Standard (ASVS)
- SANS Top 25

Conclusion

The OWASP Top 10 2021 reflects the evolving threat landscape facing web applications today. Organizations must adopt a comprehensive approach to application security that includes secure design principles, regular security testing, and continuous monitoring. The five vulnerabilities detailed in this report represent critical areas requiring immediate attention in any security program.

Success in mitigating these risks requires commitment from leadership, proper resource allocation, and integration of security throughout the software development lifecycle. Regular assessment against the OWASP Top 10 provides organizations with a baseline for measuring and improving their application security posture.

References

1. OWASP Foundation. (2021). OWASP Top 10 2021. Retrieved from <https://owasp.org/Top10/>
 2. OWASP Foundation. Application Security Verification Standard
 3. NIST Special Publication 800-53: Security and Privacy Controls
 4. CWE Top 25 Most Dangerous Software Weaknesses
 5. SANS Institute Security Training Materials
-

Report prepared for Security Assignment

Date: July 2025

Classification: Educational Use